AUDIT REPORT

Burnt Labs

Trade-OS Cosmwasm

Prepared by SCV-Security

On 19/03/2026

Table of Contents

Table of Contents 2
Introduction 3
Scope Functionality 3
Submitted Codebase 3
Methodologies 4
Code Criteria 5
Findings Summary 6
Findings Technical Details 7
1. Migration allows version downgrades 7
2. Missing overflow-checks in Cargo.toml release profile 8
3. Missing balance check before transfer 9
4. The verifier_pubkey_len attribute will always emit 33 10
5. Unused Unauthorized error variant 11
Document Control 12
Appendices 13
A. Appendix - Risk assessment methodology 13
B. Appendix - Report Disclaimer 14

Introduction

SCV has been engaged by Burnt Labs to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

TradeOS is a signature-verified claim vault deployed on Xion. Users submit claims (containing the asset, recipient, value, deadline, and comment) that are signed by an off-chain verifier. The contract validates the signature against a stored secp256k1 public key, marks the claim as consumed, and dispatches the token transfer.

Submitted Codebase

Trade-OS Contracts	
Repository	https://github.com/burnt-labs/tradeos-cw-sc
Commit	`8012d6a42f7a75df796674bf1b609eabc17d26bb`
Branch	`main`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to Burnt Labs. Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**SUFFICIENT**	N/A
Coverage	**SUFFICIENT**	N/A
Readability	**SUFFICIENT**	N/A
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
Migration allows version downgrades	LOW	PENDING
Missing `overflow-checks` in `Cargo.toml` release profile	LOW	PENDING
Missing balance check before transfer	INFO	PENDING
The `verifier_pubkey_len` attribute will always emit 33	INFO	PENDING
Unused `Unauthorized` error variant	INFO	PENDING

Findings Technical Details

1. Migration allows version downgrades

RISK IMPACT: LOW	STATUS: PENDING

Description

In `src/execute.rs:246-264`, the `migrate()` function verifies that the contract name matches, but does not check that the new version is strictly greater than the old version. This allows an admin to downgrade the contract to a previous version, potentially re-introducing patched vulnerabilities.

Recommendation

Consider adding a semver comparison to reject migration downgrades.

2. Missing `overflow-checks` in `Cargo.toml` release profile

RISK IMPACT: LOW	STATUS: PENDING

Description

The `Cargo.toml` does not include a `[profile.release]` section with `overflow-checks = true`. While the standard `cosmwasm/optimizer` Docker image does enable overflow checks by default, explicitly setting this is a best practice to guard against non-standard build environments.

Recommendation

Consider adding `overflow-checks = true` to `Cargo.toml`.

3. Missing balance check before transfer

RISK IMPACT: INFORMATIONAL	STATUS: PENDING

Description

In `Vault.sol:67-74`, the Solidity contract explicitly checks its balance before transferring funds. If there are insufficient funds, a custom `InsufficientBalance` error will be returned.

The issue is that the CosmWasm contract implementation diverges from this. In the `exec_claim` function, if the contract lacks sufficient funds, the transaction will fail at the chain level with an opaque wasm error, making debugging harder for integrators.

Recommendation

Consider returning a descriptive `InsufficientBalance` error message if the contract does not hold sufficient balance to transfer funds.

4. The `verifier_pubkey_len` attribute will always emit 33

RISK IMPACT: INFORMATIONAL	STATUS: PENDING

Description

In `src/execute.rs:38` and 92, the `instantiate` and `exec_set_verifier` functions emit an attribute `verifier_pubkey_len` as `pk.len()`.

Since both functions reject any key that is not 33 bytes (see lines 30-31 and 86-87), this attribute will always be 33. This does not convey any useful information for indexers.

Recommendation

Consider removing the attribute entirely or replacing it with something useful, such as the actual pubkey hex.

5. Unused `Unauthorized` error variant

RISK IMPACT: INFORMATIONAL	STATUS: PENDING

Description

In `src/error.rs:11`, the `ContractError::Unauthorized` variant is never constructed anywhere in the codebase. All authorization checks use `cw_ownable::assert_owner()`, which returns `OwnershipError` and is auto-converted to `ContractError::Ownership`.

Recommendation

Consider removing the unused variant.

Document Control

Version	Date	Notes
-	6th March 2026	Security audit commencement date.
0.1	19th March 2026	Initial report with identified findings delivered.
0.5	-	Fixes remediations implemented and reviewed.
1.0	-	Audit completed, final report delivered.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING

SCV SECURITY

scv.services

contact@scv.services